CAF — API & Integrations

Aggregated export — 2026-06-16. Source markdown in repo is unchanged.

Bundle id: `06-caf-api-and-integrations`

Included: docs/API_REFERENCE.md

CAF Core HTTP API

Base URL: `http://localhost:3847` (or your deploy).
Auth: if enabled, send `x-caf-core-token: <CAF_CORE_API_TOKEN>` or `Authorization: Bearer <token>` on all routes **except**:

- `GET /health`
- `GET /readyz`
- `GET /health/rendering`
- `GET /robots.txt`
- public renderer-template reads (see `src/routes/renderer-templates.ts` and `src/server.ts`)

`GET /health`

```
{ "ok": true, "service": "caf-core", "engine_version": "v1" }
```

`POST /v1/decisions/plan`

Computes suppression, scores, ranking, prompt/route selection; persists `decision_traces` unless `dry_run: true` .

Body

```
{
  "project_slug": "SNS",
```

```
  "run_id": "SNS_2026W09",
  "dry_run": false,
  "min_score": 0.4,
  "max_candidates": 20,
  "max_variations_per_candidate": 2,
  "prompt_override": {
    "template_only": false,
    "prompt_id": "carousel_v2",
    "prompt_version_id": "optional-uuid"
  },
  "candidates": [
    {
      "candidate_id": "SNS_2026W09_Instagram_0001",
      "flow_type": "FLOW_CAROUSEL",
      "platform": "Instagram",
      "target_platform": "Instagram",
      "confidence_score": 0.72,
      "platform_fit": 0.8,
      "novelty_score": 0.55,
      "past_performance_similarity": 0.6,
      "recommended_route": "HUMAN_REVIEW",
      "content_idea": "Hook about weekly reset",
      "dedupe_key": "optional-stable-key"
    }
  ]
}
```

Response (`200`): `{ "ok": true, "result": { "trace_id", "selected", "dropped_candidates", "suppression_reasons", "meta", ... } }`

`POST /v1/jobs/ingest`

```
{
  "project_slug": "SNS",
  "task_id": "SNS_2026W09__Instagram__FLOW_CAROUSEL__row0001__v1",
  "run_id": "SNS_2026W09",
  "candidate_id": "SNS_2026W09_Instagram_0001",
  "variation_name": "v1",
  "flow_type": "FLOW_CAROUSEL",
  "platform": "Instagram",
  "status": "PLANNED",
```

```
  "recommended_route": "HUMAN_REVIEW",
  "pre_gen_score": 0.61,
  "generation_payload": { "caption": "..." }
}
```

`GET /v1/jobs/:project_slug/:task_id`

Returns `{ "ok": true, "job": { ...row } }` or `404`.

`PUT /v1/projects/:project_slug/constraints`

```
{
  "max_daily_jobs": 80,
  "min_score_to_generate": 0.35,
  "max_active_prompt_versions": 3,
  "default_variation_cap": 2,
  "auto_validation_pass_threshold": 0.72
}
```

`null` allowed for numeric caps you want unset (e.g. unlimited daily jobs).

`POST /v1/prompt-versions`

```
{
  "project_slug": "SNS",
  "flow_type": "FLOW_CAROUSEL",
  "prompt_id": "carousel_v2",
  "version": "2.1.0",
  "status": "active",
  "temperature": 0.7,
  "max_tokens": 2000
}
```

`POST /v1/suppression/rules`

```
{
  "project_slug": "SNS",
  "name": "High rejection carousel",
  "rule_type": "REJECTION_RATE",
  "scope_flow_type": "FLOW_CAROUSEL",
  "threshold_numeric": 0.55,
  "window_days": 14,
  "action": "BLOCK_FLOW"
}
```

`rule_type` : `REJECTION_RATE` | `QC_FAIL_RATE` | `ENGAGEMENT_FLOOR` | `BLOCK_FLOW` | `BLOCK_PROMPT_VERSION`
`action` : `BLOCK_FLOW` | `REDUCE_VOLUME` | `FORCE_HUMAN_REVIEW` | `BLOCK_PROMPT_VERSION`

`POST /v1/learning/rules`

```
{
  "project_slug": "SNS",
  "rule_id": "rule_reject_soft_hooks_01",
  "trigger_type": "REJECT_RATE",
  "scope_flow_type": "FLOW_CAROUSEL",
  "action_type": "BOOST_RANK",
  "action_payload": { "multiplier": 1.02 },
  "confidence": 0.8,
  "source_entity_ids": ["task_123"]
}
```

Apply (sets `applied_at` , `status = active`):
`POST /v1/learning/rules/:project_slug/:rule_id/apply`

List: `GET /v1/learning/rules/:project_slug`

POST /v1/transitions

```
{
  "project_slug": "SNS",
  "task_id": "SNS_2026W09__Instagram__FLOW_CAROUSEL__row0001__v1",
  "from_state": "GENERATED",
  "to_state": "IN_REVIEW",
  "triggered_by": "system",
  "metadata": {}
}
```

`triggered_by` : `system` | `human` | `rule` | `experiment`

POST /v1/audits

```
{
  "project_slug": "SNS",
  "task_id": "SNS_2026W09__Instagram__FLOW_CAROUSEL__row0001__v1",
  "audit_type": "diagnostic",
  "failure_types": ["generic_hook"],
  "audit_score": 0.42,
  "metadata": {}
}
```

POST /v1/reviews

```
{
  "project_slug": "SNS",
  "task_id": "SNS_2026W09__Instagram__FLOW_CAROUSEL__row0001__v1",
  "decision": "REJECTED",
  "rejection_tags": ["off_brand"],
  "validator": "you@example.com",
```

```
  "submit": true
}
```

`POST /v1/metrics`

Use `metric_window` : `stabilized` for learning-driving metrics.

```
{
  "project_slug": "SNS",
  "task_id": "SNS_2026W09__Instagram__FLOW_CAROUSEL__row0001__v1",
  "platform": "Instagram",
  "metric_window": "stabilized",
  "window_label": "72h",
  "engagement_rate": 0.041,
  "likes": 1200,
  "saves": 90
}
```

`POST /v1/auto-validation`

```
{
  "project_slug": "SNS",
  "task_id": "SNS_2026W09__Instagram__FLOW_CAROUSEL__row0001__v1",
  "hook": "Sunday reset ritual",
  "caption": "Long caption …",
  "banned_substrings": ["guaranteed", "cure"]
}
```

Returns heuristic scores and inserts `auto_validation_results` .

Handlers: *`src/routes/v1.ts`* .

Included: docs/VIDEO_FLOWS.md

CAF Core video flows (how they work)

This doc explains how **video jobs** are routed and rendered in CAF Core, with special focus on **HeyGen duration constraints**, **subtitle behavior**, and **avatar/voice selection**.

HeyGen HTTP API (v3) reference (endpoints, auth, Video Agent vs direct video): `HEYGEN_API_V3.md` .

Glossary

- **Job**: a row in `caf_core.content_jobs` identified by `task_id` (primary execution key).
- **generated_output**: `content_jobs.generation_payload.generated_output` (LLM outputs and plans).
- **Scene pipeline**: multi-scene clip concat + optional TTS + subtitle burn-in.
- **HeyGen pipeline**: single-take (or agent-driven) HeyGen generation.

Planning: which video flow_type is chosen at run start

When `project_system_constraints.video_routing.enabled` is true (default after migration `059`):

1. Planner rows with `format: "video"` are routed to **one** enabled flow: `script_avatar` → `FLOW_VID_SCRIPT` , `prompt_avatar` → `FLOW_VID_PROMPT` , `no_avatar` → `FLOW_VID_PROMPT_NO_AVATAR` (plus legacy aliases and `FLOW_PRODUCT_*` via `heygen_mode`).
2. Set `video_style` on ideas (`script_avatar` | `prompt_avatar` | `no_avatar`) when materializing candidates; b-roll / multi-scene ideas should use `no_avatar` (scene assembly is **not** used by this router).
3. **Scene assembly** flows are excluded from expansion while routing is on; the scene-assembly LLM router is skipped.

Implementation: `src/decision_engine/video-flow-routing.ts` , `src/services/run-orchestrator.ts` (`buildCandidatesFromSignalPack`).

Routing: how a job becomes a rendered video

Entry points

- `POST /v1/jobs/:project_slug/:task_id/process` → `processJobByTaskId(...)` in `src/routes/runs.ts`

- `POST /v1/runs/:project_slug/:run_id/process` → starts background **draft package generation** (`generateRunDraftPackages(...)`) in `src/routes/runs.ts` (LLM → QC → diagnostics; jobs stop at `GENERATED`). Rendering is a separate operator step via `POST /v1/runs/:project_slug/:run_id/render` .

Video decision point

All video-related work routes through `processVideoJob(...)` in `src/services/job-pipeline.ts` .

The routing logic is (conceptually):

1. **Scene pipeline** if any of the following are true:

- the chosen production route includes `SCENE` , or
- `generated_output.scene_bundle` exists, or
- `generation_payload.video_pipeline` is `"scene"` , or
- `flow_type` matches `Video_Scene_Generator` / `FLOW_SCENE_ASSEMBLY` / `scene_assembly` patterns

2. Else **HeyGen pipeline** if `HEYGEN_API_KEY` is configured:

- pre-step: `ensureHeygenPayloadForFlowType(...)` (adds missing script/prompt fields to `generated_output`)
- render: `runHeygenForContentJob(...)` in `src/services/heygen-renderer.ts`

3. Else fallback to **remote video-assembly** `/full-pipeline`

Key files:

- `src/services/job-pipeline.ts` (routing + orchestration)
- `src/decision_engine/flow-kind.ts` (`isVideoFlow(...)` heuristic classifier)

Scene pipeline (multi-scene): clips → concat → TTS → subtitles → mux

Main orchestrator: `runScenePipeline(...)` in `src/services/scene-pipeline.ts` .

1) Scene bundle preparation

The pipeline ensures `generated_output.scene_bundle.scenes[]` exists via:

- `ensureSceneBundleInPayload(...)` in `src/services/scene-assembly-generator.ts`

Scene bundles typically contain:

- `scenes[i].video_prompt` (visual prompt per scene)
- optional `scenes[i].scene_narration_line` (text slice of spoken script aligned to that scene)
- optional clip URLs (`rendered_scene_url` , etc.)

2) Clip rendering (optional)

If scene clips are missing but prompts exist, Core can generate clips:

- **Sora (OpenAI Videos API)** when `SCENE_ASSEMBLY_CLIP_PROVIDER=sora`
  - implemented in `src/services/sora-scene-clips.ts`
  - requires `OPENAI_API_KEY` and Supabase config so clips can be uploaded to fetchable URLs
- **HeyGen Video Agent fallback** when `SCENE_ASSEMBLY_CLIP_PROVIDER=heygen` and `SCENE_ASSEMBLY_HEYGEN_CLIP_FALLBACK=1`
  - implemented in `src/services/scene-pipeline.ts` using helpers from `src/services/heygen-renderer.ts`
  - runs in **no-avatar** mode for clip segments

3) Concat (video-assembly)

Scene MP4 URLs are passed to the Node video-assembly service:

- `POST /concat-videos?async=1` with `{ video_urls, task_id, run_id }`
- poll `GET /status/:request_id`

Implementation:

- CAF Core client: `src/services/scene-pipeline.ts` (`pollVideoAssemblyJob(...)`)
- video-assembly server: `services/video-assembly/server.js`

4) TTS voiceover (optional)

If `generated_output.spoken_script` exists and `OPENAI_API_KEY` is configured, the scene pipeline synthesizes voiceover:

- `synthesizeSpeechToStorage(...)` in `src/services/tts-service.ts`

Script length enforcement in the scene pipeline (optional)

The scene pipeline can optionally **trim `spoken_script` to fit the clip timeline**:

- Controlled by `SCENE_ENFORCE_SPOKEN_SCRIPT_WORD_TRIM` (boolean)
- Budget computed from:
  - timeline seconds (`clipDurs`)
  - `SCENE_VO_WORDS_PER_MINUTE`
  - `SCENE_VO_WORD_BUDGET_SAFETY`
- Trimming performed by `fitSpokenScriptToWordBudget(...)` in `src/services/spoken-script-word-budget.ts`

If trimming occurs, the pipeline writes the shortened script back into `generated_output` (`spoken_script` and `script`).

5) Subtitles (SRT generation) and how they're picked

Scene pipeline subtitles are **generated from text**, not from transcription:

Source selection order (in `src/services/scene-pipeline.ts`):

1. Use `scene_narration_line` if it aligns exactly with the `spoken_script` (strict), checked by:
   - `narrationLinesAlignedWithScript(...)` in `src/services/scene-narration-alignment.ts`
2. Else accept a looser concatenation alignment:
   - `narrationLinesLooseConcatMatchesScript(...)`
3. Else split the `spoken_script` into per-scene chunks weighted by clip durations:
   - `splitScriptIntoSceneChunksByWeights(...)` in `src/services/caption-generator.ts`

SRT is built via:

- `buildSrtFromScenesWithSentenceCues(...)` in `src/services/caption-generator.ts`

The resulting file is uploaded to:

- `subtitles/{run}/{task}/captions.srt`

6) Mux + burn subtitles (video-assembly)

Final mux is done by video-assembly:

- `POST /mux?async=1` with `{ video_url, audio_url, subtitles_url? }`

If `subtitles_url` is provided, video-assembly downloads the SRT and runs ffmpeg with a subtitles filter to burn them into the MP4.

Implementation:

- CAF Core: `src/services/scene-pipeline.ts` (signs URLs, calls mux)
- video-assembly: `services/video-assembly/server.js` (`muxAudioBurnSubtitles(...)`)

HeyGen pipeline (single-take): request building, duration constraints, captions

Main orchestrator: `runHeygenForContentJob(...)` in `src/services/heygen-renderer.ts` .

Script and prompt fields

CAF Core extracts text from `generated_output` using aliases/nesting:

- `extractSpokenScriptText(...)` and `extractVideoPromptText(...)` in `src/services/video-gen-fields.ts`

Depending on the HeyGen path (see `HEYGEN_API_V3.md` for upstream API details):

- **HeyGen v3 direct avatar video** (`POST /v3/videos` , `type: "avatar"`):
  - Built from the same merged `video_inputs` shape as the old v2 path, then mapped to a flat v3 body (`avatar_id` , `script` , `voice_id` , `aspect_ratio` , optional `callback_url` , etc.).
  - Used for **script-led** avatar jobs (`Video_Script_*` HeyGen flows).
- **HeyGen v3 Video Agent** (`POST /v3/video-agents`):
  - The `prompt` is built by `buildHeyGenVideoAgentProductionBrief` in `src/services/heygen-video-agent-prompt.ts` : a structured **CAF VIDEO AGENT PRODUCTION BRIEF** (objective, delivery mode, script/VO rules, scaled scene plan, visual style bullets, media-type guidance, on-screen text

rules, post-only caption/hashtag context). Product jobs append brand constraints (before product facts) from `product-video-agent-brand.ts` / `product-video-agent-product.ts`.
    - Optional `avatar_id`, `voice_id`, `style_id`, `orientation`, `callback_url` are passed when configured (`style_id` from `heygen_config` keys `style_id` or `heygen_style_id`).
    - Used for **prompt-led** avatar jobs and **no-avatar** agent jobs. Canonical `flow_type` values: `FLOW_VID_PROMPT` (Video Agent, avatar from `heygen_config`), `FLOW_VID_PROMPT_NO_AVATAR` (Video Agent, no on-camera avatar), `FLOW_VID_SCRIPT` (direct `/v3/videos` when `HEYGEN_AVATAR`), plus legacy `Video_Prompt_*` / `Video_Script_*` names.
- **Legacy v2** (`POST /v2/video/generate`) — **only** when the job uses HeyGen `voice: { type: "silence" }` (visual-only / no spoken script). The v3 create-video schema has no silence-TTS equivalent, so CAF keeps this one legacy call for that edge case.

Duration constraints: what is enforced vs guidance

Important: HeyGen **avatar** `POST /v3/videos` has no `duration_sec` — runtime follows TTS. CAF enforces **word budgets** so "target 30–60s" is not only prompt text:

- **Default:** `HEYGEN_ENFORCE_SPOKEN_SCRIPT_WORD_BOUNDS=true` (env). Min/max words = `VIDEO_TARGET_DURATION_{MIN,MAX}_SEC` × `SCENE_VO_WORDS_PER_MINUTE` / 60 (see `heygenSpokenScriptWordBoundsFromConfig` in `src/services/spoken-script-word-budget.ts`).
- **Before HeyGen:** `enforceHeygenSpokenScriptWordLaw` in `src/services/heygen-spoken-script-enforcement.ts` trims over-max scripts, expands under-min via OpenAI when `OPENAI_API_KEY` is set, or **fails** with a clear error.
- **Script prep LLM:** `ensureVideoScriptInPayload` retries once if the first draft is under the minimum word count (`src/services/video-script-generator.ts`).

What *is* also enforced for **Video Agent**:

- CAF Core clamps a target length in seconds, then embeds it in the agent prompt (e.g. "Target duration: about N seconds"). The v3 `POST /v3/video-agents` body is normalized and may omit `duration_sec` per HeyGen schema.

- `resolveHeygenAgentDurationSec(...)` in `src/services/heygen-renderer.ts`
- bounds include `HEYGEN_AGENT_MIN_DURATION_SEC` and a max of 300s (hard-coded), and a fallback based on `VIDEO_TARGET_DURATION_MIN_SEC`.

Additional **guidance** (LLM):

- Prompts are told to target a duration band via:
  - `appendVideoUserPromptDurationHardFooter(...)` and `withVideoScriptDurationPolicy(...)` in `src/services/video-content-policy.ts`

Subtitles / captions for HeyGen

Scope: script-led `/v3/videos` only. Video Agent (`/v3/video-agents`) and silence-voice (`/v2/video/generate`) jobs are intentionally excluded — Video Agent prompts produce non-script narration whose words/timing aren't knowable client-side (a synthesized SRT wouldn't match what the avatar actually said), and silence-voice has nothing to caption.

HeyGen v3 `POST /v3/videos` does **not** burn captions into the MP4 — there is no v3 caption-burn parameter on the create endpoint, and `captioned_video_url` is only populated by the video-translation proofread workflow. CAF therefore does the burn locally for script-led jobs:

1. **Request side:** `mapHeyGenV2StyleBodyToV3CreateVideoAvatar(...)` injects `caption: { file_format: "srt" }` (per HeyGen v3 OpenAPI `CaptionSetting`) so HeyGen renders an **SRT sidecar** at `data.subtitle_url`. The MP4 is unchanged.
2. **Status surface:** `pickHeyGenDownloadUrlFromStatus(...)` returns `{ url, usedVideoUrlCaption, subtitleUrl, durationSec }`. CAF still prefers `captioned_video_url` / `video_url_caption` when present (legacy / proofread paths) and skips burning if HeyGen already burned in.
3. **Burn step:** `runHeygenForContentJob(...)` calls `maybeBurnHeygenSubtitles(...)`. The function gates on `postPath === "/v3/videos"` first; if not script-led, it short-circuits with `subtitles_burn_skipped_reason: "script_led_only (postPath=...)"`. For script-led jobs it downloads HeyGen's SRT (or, only when HeyGen omits one, synthesizes via `buildRoughSrt(spoken_script, durationSec)`), uploads it + signs the stored MP4, then `POST {VIDEO_ASSEMBLY_BASE_URL}/burn-subtitles?async=1` → polls →

downloads the burned MP4 → **overwrites the same Supabase object path** so the canonical asset includes captions.

4. **video-assembly:** `services/video-assembly/server.js` exposes `POST /burn-subtitles` (added alongside `/mux`) — copies audio with `-c:a copy` and re-encodes video with `libx264 + subtitles` filter.

Config keys (in `src/config.ts`):

- `HEYGEN_BURN_SUBTITLES` (default `true`): set to `0` / `false` to keep the raw HeyGen MP4 (no captions).
- `HEYGEN_BURN_SUBTITLES_POLL_MAX_MS` (default 900000): how long to wait on the burn job.
- `HEYGEN_BURN_SUBTITLE_FORCE_STYLE` (optional): ffmpeg `force_style` override for HeyGen burns only.

The resulting `assets` row records `metadata_json.subtitles_burned`, `subtitles_source` (`heygen_v3_srt` | `synthesized_from_spoken_script`), `subtitles_burn_skipped_reason`, and `subtitles_burn_error` for auditability. The burn step is best-effort: if it fails or is skipped, the raw HeyGen MP4 remains the stored asset and the failure is captured in `api_call_audit` (step `heygen_burn_subtitles`).

Avatar + voice selection (HeyGen): precedence and config keys

Where config is stored

Per-project HeyGen configuration is stored as key/value rows:

- Postgres table: `caf_core.heygen_config` (see `migrations/002_project_config_and_runs.sql`)
- Accessors: `listHeygenConfig(...)` / `upsertHeygenConfig(...)` in `src/repositories/project-config.ts`

Rows can be scoped by optional fields:

- `platform` (nullable; null means “all platforms”)
- `flow_type` (nullable; null means “all flows”)
- `render_mode` (nullable; null means “all render modes”)

How rows merge for a job

`mergeHeygenConfigForJob(...)` in `src/services/heygen-renderer.ts` merges all matching rows, with wildcards allowed.

Compatibility note: alternate render-mode spellings `PROMPT` / `SCRIPT` (from admin or imported config rows) are treated as compatible with job render modes `HEYGEN_AVATAR` / `HEYGEN_NO_AVATAR` depending on the flow type.

Avatar selection

Avatar can come from either:

- **Pools** (preferred): `prompt_avatar_pool_json`, `script_avatar_pool_json`, `avatar_pool_json` \n Each is a JSON array of objects like `{ \"avatar_id\": \"...\", \"voice_id\": \"...\" }` (voice_id optional).
- **Single IDs**: `prompt_avatar_id`, `script_avatar_id`, `avatar_id`

Pool picks are deterministic **and round-robin within a run**:

- Seed defaults to `task_id` (scene-assembly overrides to `${task_id}__scene_${i}`).
- `stablePickIndex(seed, pool.length)` parses `row{NNNN}` / `scene_{i}` / `v{N}` from the seed and computes `((row - 1) + (variation - 1) + scene) mod pool.length`. So:
  - Within a run, jobs `row0001`, `row0002`, `row0003` pick entries `0`, `1`, `2`, then wrap.
  - Within a multi-scene job, scenes `0`, `1`, `2` rotate independently.
  - Variations `v1`, `v2` of the same row also rotate (so they don't collide on a single avatar).
- Same `(row, scene, v)` triple → same index, so retries / restarts always re-pick the same `(avatar_id, voice_id)` pair from the pool.
- Ad-hoc seeds with no `row` / `scene` / `v` markers fall back to a stable FNV-like hash pick.

No-avatar flows skip avatar injection entirely.

Voice selection

Voice resolution order (simplified):

1. If an avatar pool entry includes `voice_id`, use it.
2. Else use config keys like `voice`, `voice_id`, `default_voice`, `default_voice_id`.
3. For script-led flows, `script_voice_id` can override.
4. Else fallback to environment `HEYGEN_DEFAULT_VOICE_ID`.
5. Else final hard fallback constant in `src/services/heygen-renderer.ts`.

Operational notes / debugging

- Most pipeline steps insert `api_call_audit` rows via `tryInsertApiCallAudit(...)` to capture request/response metadata per `task_id`.
- For HeyGen generate (`step` `heygen_video_generate`), audit `requestJson` includes `request_body_pre_normalization` and `request_body_sent` so you can compare CAF-internal fields (e.g. `duration_sec`) with the normalized JSON actually POSTed to `/v3/video-agents`.
- For HeyGen, the resulting `assets` row includes `metadata_json.heygen_used_video_url_caption` so you can confirm whether captioned output was used.
- For scene pipeline, `content_jobs.scene_bundle_state` is updated with a compact report, warnings, and mux details.

Included: docs/HEYGEN_API_V3.md

HeyGen External API (v3) — workspace reference

Consolidated notes for integrating with HeyGen. Official human docs: docs.heygen.com. CAF Core wiring for jobs lives in `VIDEO_FLOWS.md` and `src/services/heygen-renderer.ts`.

CAF Core usage (implementation)

- **Poll:** `GET /v3/videos/{video_id}` first; on **404** only, fall back to legacy `GET .../v1/video_status.get?video_id=` for older ids.
- **Video Agent:** `POST /v3/video-agents` with a normalized body (unknown keys stripped; target length lives in the `prompt` text, not a `duration_sec` field).
- **API audit (`api_call_audit`, step `heygen_video_generate`):** `requestJson` stores both `request_body_pre_normalization` (CAF-internal shape, may include `duration_sec` for debugging) and `request_body_sent` (payload after `normalizeHeyGenSubmitPayload`, i.e. what was actually stringified for `fetch`). This makes it obvious when a field appears in audit but not on the wire.
- **Script + avatar:** `POST /v3/videos` with `{ type: "avatar", avatar_id, script, voice_id, aspect_ratio, ... }` mapped from the internal `video_inputs[0]` builder.

- **Silence / visual-only (no spoken script):** still `POST /v2/video/generate` when `voice.type === "silence"` — v3 create-video has no equivalent.
- **Captions on `POST /v3/videos` (script-led only):** v3 has no caption-burn parameter. For script-led avatar jobs CAF passes `caption: { file_format: "srt" }` (per HeyGen v3 OpenAPI `CaptionSetting`) so HeyGen renders an **SRT sidecar** at `data.subtitle_url` — *the MP4 itself is not modified*. CAF then burns the SRT in locally via the video-assembly `POST /burn-subtitles` endpoint and replaces the stored MP4 in Supabase. Toggle with `HEYGEN_BURN_SUBTITLES=0` to keep the raw HeyGen MP4. **Video Agent (`/v3/video-agents`) and silence-voice (`/v2/video/generate`) jobs are intentionally excluded from the burn step** — Video Agent narration is generated server-side and not knowable client-side, so a synthesized SRT wouldn't match the spoken audio.
- **Captioned download (legacy / proofread workflows):** prefer `data.captioned_video_url`, then legacy `video_url_caption`, then `video_url`. v3 only populates `captioned_video_url` for video-translation proofread renders; the regular `POST /v3/videos` create flow does not produce a burned-in MP4.

Documentation index (for LLMs / discovery)

Fetch the full page list before deep exploration:

- **Index URL:** `https://heygen-1fa696a7.mintlify.app/llms.txt`

Base URL and auth

Item	Value
Production base	`https://api.heygen.com`
API key header	`x-api-key` (OpenAPI name `ApiKeyAuth`; examples often use `X-Api-Key`)
OAuth	`Authorization: Bearer <token>` (`BearerAuth`)

Obtain API keys from the HeyGen dashboard (Settings → API). Environment variable used in docs: `HEYGEN_API_KEY`.

Versioning

Legacy `/v1` and `/v2` remain supported until **October 31, 2026**. New capabilities (CLI, MCP, Voice design API, improved errors, latest models such as lipsync) are **v3-only**. Prefer v3 for new work.

Video Agent vs direct video creation

	Video Agent	Direct video
Endpoint	`POST /v3/video-agents`	`POST /v3/videos`
Input	Natural language prompt	Structured JSON
Script	Agent writes	You supply
Avatar / voice	Agent picks (optional overrides)	You specify
Interactive iteration	Yes (`mode: "chat"`)	No
Webhooks	`callback_url`	`callback_url`
Control	Lower (prompt-driven)	Higher (explicit)

Practical guidance: start with Video Agent; switch to `POST /v3/videos` when you need fixed avatar, voice, and script.

Endpoints (summarized from OpenAPI excerpts)

`GET /v3/video-agents/styles` — list Video Agent styles

Curated visual style templates (`style_id` for `POST /v3/video-agents`).

Query: `tag` (optional filter, e.g. `cinematic` , `retro-tech`), `limit` (1–100, default 20), `token` (pagination cursor).

200 body: `data[]` (`StyleItem`), `has_more` , `next_token` .

`StyleItem` : `style_id` , `name` , optional `thumbnail_url` , `preview_video_url` , `tags` , `aspect_ratio` .

`GET /v3/video-agents` — list Video Agent sessions

Paginated sessions, newest first.

Query: `limit` (1–100, default 20), `token` .

200 body: `data[]` (`SessionListItem` : `session_id` , optional `title` , `created_at`), `has_more` , `next_token` .

`POST /v3/video-agents` — create Video Agent session

One-shot or multi-turn video from a prompt.

Body (`CreateVideoAgentRequest`):

Field	Notes
`prompt`	**Required.** 1–10000 characters
`mode`	`generate` (default) or `chat`
`avatar_id`	Optional override
`voice_id`	Optional override
`style_id`	From `GET /v3/video-agents/styles`
`orientation`	`landscape` | `portrait` or omit (auto)
`files`	Up to 20 items; discriminated union: `{ type: "url", url }`, `{ type: "asset_id", asset_id }`, `{ type: "base64", media_type, data }`
`callback_url`	Webhook on completion/failure
`callback_id`	Optional id echoed in webhook payload
`incognito_mode`	Default `false`; disables memory injection/extraction

200 body: `data` with `session_id` , `status` (`generating` | `thinking` | `completed` | `failed`), optional `video_id` , `created_at` .

Poll video with `GET /v3/videos/{video_id}` when `video_id` is present.

`GET /v3/avatars` — list avatar groups

Paginated avatar groups (characters); each group has one or more **looks**. Use `GET /v3/avatars/looks` (per HeyGen docs) for look-level detail and engine compatibility.

Query: `ownership` (`public` | `private` or omit for all), `limit` (1–50, default 20), `token` .

`AvatarGroupItem` highlights: `id` , `name` , `created_at` , `looks_count` , optional previews, `gender` , `default_voice_id` , `consent_status` , optional `status` (`processing` | `pending_consent` | `failed` | `completed`) for private avatars, optional `error` when failed.

`POST /v3/assets` — upload asset

Multipart `file` field. **Max 32 MB.** Types: png, jpeg, mp4, webm, mp3, wav, pdf.

200 `data` : `asset_id` , `url` , `mime_type` , `size_bytes` . Use `asset_id` in Video Agent `files` or other v3 flows.

Quick start (async flow)

1. **Create session:** `POST https://api.heygen.com/v3/video-agents` with JSON `{ "prompt": "..." }` and `X-Api-Key` .
2. **Read** `data.video_id` from the response (may be null in some multi-turn cases).
3. **Poll** `GET https://api.heygen.com/v3/videos/{video_id}` until `status` is `completed` or `failed` .
4. Typical video status progression: `pending` → `processing` → `completed` | `failed` . Completed responses include `video_url` , `thumbnail_url` , `duration` , etc.

Webhook: pass `callback_url` on create to avoid polling.

Common errors

Errors use a wrapper with `error: { code, message, param?, doc_url? }` . Examples:

- `invalid_parameter` (e.g. bad `limit`)
- `authentication_failed`
- `rate_limit_exceeded` (respect `Retry-After` header when present)

Cookbook pattern: personalized outreach (batch)

1. Build a prompt template with prospect-specific variables (name, company, role, pain, value prop).
2. `POST /v3/video-agents` per prospect; space requests for rate limits (e.g. sleep between calls).
3. Poll each `video_id` (or use webhooks); deliver `video_url` / `thumbnail_url` via email, LinkedIn, etc.

Brand consistency: reuse the same `avatar_id` , `voice_id` , and style instructions across batches.

Related HeyGen surfaces

- HeyGen Developers — overview, pricing pointers, Video Agent / Translation / Lipsync / Voices.

- Dashboard API key: Settings → API (linked from HeyGen quick start).
- CLI and MCP are documented on the developer site for agent-style workflows.

This file is a project-local reference; always verify request/response shapes against the current HeyGen OpenAPI or official docs if something fails at runtime.